

Praktikum 1 - Struktur Sistem Operasi

Daftar Isi

1. [Operating-System Services](#)
 2. [User and OS Interface](#)
 3. [System Calls](#)
 4. [System Services](#)
 5. [Linkers and Loaders](#)
 6. [OS-Specific Applications](#)
 7. [OS Design and Implementation](#)
 8. [OS Structure](#)
 9. [Building and Booting OS](#)
 10. [OS Debugging](#)
-

1. Operating-System Services

Praktik 1.1: Menjelajahi Layanan OS

```
# Lihat informasi sistem
uname -a
lsb_release -a

# Lihat proses yang berjalan
ps aux | head -20

# Lihat penggunaan memori
free -h

# Lihat informasi CPU
lscpu

# Lihat perangkat yang terpasang
lsblk
```

Praktik 1.2: Program C - Menggunakan Layanan OS

Buat file `os_services.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <time.h>
```

```
int main() {
    // 1. Program Execution Service
    printf("=== Program Execution Service ===\n");
    printf("Process ID (PID): %d\n", getpid());
    printf("Parent Process ID (PPID): %d\n", getppid());

    // 2. I/O Operations Service
    printf("\n=== I/O Operations Service ===\n");
    FILE *fp = fopen("test_file.txt", "w");
    if (fp != NULL) {
        fprintf(fp, "Testing I/O operations\n");
        fclose(fp);
        printf("File created and written successfully\n");
    }

    // 3. File System Manipulation
    printf("\n=== File System Manipulation ===\n");
    struct stat file_stat;
    if (stat("test_file.txt", &file_stat) == 0) {
        printf("File size: %ld bytes\n", file_stat.st_size);
        printf("File permissions: %o\n", file_stat.st_mode & 0777);
    }

    // 4. Communications
    printf("\n=== Communications ===\n");
    printf("Hostname: %s\n", getenv("HOSTNAME"));

    // 5. Error Detection
    printf("\n=== Error Detection ===\n");
    FILE *fp2 = fopen("nonexistent.txt", "r");
    if (fp2 == NULL) {
        perror("Error opening file");
    }

    // 6. Resource Allocation
    printf("\n=== Resource Allocation ===\n");
    int *ptr = (int*)malloc(100 * sizeof(int));
    if (ptr != NULL) {
        printf("Memory allocated successfully\n");
        free(ptr);
    }

    return 0;
}
```

Compile dan jalankan:

```
gcc -o os_services os_services.c
./os_services
```

2. User and Operating-System Interface

Praktik 2.1: Command Line Interface (CLI)

```
# Navigasi dasar
pwd
ls -la
cd /tmp
mkdir test_dir
cd test_dir

# Pipelining
ls -l | grep "^d"
ps aux | grep bash | wc -l

# Redirection
echo "Hello World" > output.txt
cat output.txt
cat output.txt >> output.txt
cat < output.txt

# Background processes
sleep 30 &
jobs
fg %1
```

Praktik 2.2: Program C - Shell Sederhana

Buat file `simple_shell.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, MAX_INPUT, stdin) == NULL) {
            break;
        }
    }
}
```

```

    }

    // Hapus newline
    input[strcspn(input, "\n")] = 0;

    // Exit command
    if (strcmp(input, "exit") == 0) {
        break;
    }

    // Parse command
    int argc = 0;
    char *token = strtok(input, " ");
    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }
    args[argc] = NULL;

    if (argc == 0) continue;

    // Fork dan execute
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execvp(args[0], args);
        perror("execvp failed");
        exit(1);
    } else if (pid > 0) {
        // Parent process
        wait(NULL);
    } else {
        perror("fork failed");
    }
}

return 0;
}

```

Compile dan jalankan:

```

gcc -o simple_shell simple_shell.c
./simple_shell

```

3. System Calls

Praktik 3.1: Process Control System Calls

Buat file `process_syscalls.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    printf("=== Process Control System Calls ===\n");
    printf("Parent PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        // Child process
        printf("Child PID: %d\n", getpid());
        printf("Child's Parent PID: %d\n", getppid());

        // exec system call
        printf("Child executing 'ls' command...\n");
        execlp("ls", "ls", "-l", NULL);

        // Jika exec berhasil, kode di bawah tidak akan dijalankan
        perror("exec failed");
        exit(1);
    } else {
        // Parent process
        printf("Parent waiting for child (PID: %d)...\n", pid);

        int status;
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n", WEXITSTATUS(status));
        }
    }

    return 0;
}
```

Praktik 3.2: File Management System Calls

Buat file `file_syscalls.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
```

```

#include <string.h>

int main() {
    printf("=== File Management System Calls ===\n");

    // open() system call
    int fd = open("syscall_test.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open failed");
        exit(1);
    }
    printf("File descriptor: %d\n", fd);

    // write() system call
    const char *text = "Hello from system calls!\n";
    ssize_t bytes_written = write(fd, text, strlen(text));
    printf("Bytes written: %ld\n", bytes_written);

    // close() system call
    close(fd);
    printf("File closed\n");

    // read() system call
    fd = open("syscall_test.txt", O_RDONLY);
    if (fd < 0) {
        perror("open for reading failed");
        exit(1);
    }

    char buffer[100];
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    buffer[bytes_read] = '\0';
    printf("Read from file: %s", buffer);

    close(fd);

    // unlink() system call (delete file)
    // unlink("syscall_test.txt");

    return 0;
}

```

Praktik 3.3: Information Maintenance System Calls

Buat file `info_syscalls.c`:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <time.h>
#include <sys/times.h>

```

```
int main() {
    printf("=== Information Maintenance System Calls ===\n\n");

    // getpid() and getppid()
    printf("Process ID: %d\n", getpid());
    printf("Parent Process ID: %d\n", getppid());

    // getuid() and getgid()
    printf("User ID: %d\n", getuid());
    printf("Group ID: %d\n", getgid());

    // time()
    time_t current_time = time(NULL);
    printf("Current time: %s", ctime(&current_time));

    // times()
    struct tms time_buf;
    clock_t elapsed = times(&time_buf);
    printf("Clock ticks since boot: %ld\n", elapsed);

    // sleep()
    printf("Sleeping for 2 seconds...\n");
    sleep(2);
    printf("Awake!\n");

    return 0;
}
```

Compile semua:

```
gcc -o process_syscalls process_syscalls.c
gcc -o file_syscalls file_syscalls.c
gcc -o info_syscalls info_syscalls.c

./process_syscalls
./file_syscalls
./info_syscalls
```

4. System Services

Praktik 4.1: Menggunakan systemd

```
# Lihat semua service
systemctl list-units --type=service

# Status service
systemctl status ssh
```

```
# Lihat log service
journalctl -u ssh -n 20

# Lihat service yang berjalan saat boot
systemctl list-unit-files --type=service --state=enabled
```

Praktik 4.2: Membuat Custom System Service

Buat program sederhana `my_daemon.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <syslog.h>
#include <signal.h>
#include <sys/stat.h>

void signal_handler(int sig) {
    if (sig == SIGTERM) {
        syslog(LOG_INFO, "My daemon stopped");
        closelog();
        exit(0);
    }
}

int main() {
    // Daemonize
    pid_t pid = fork();
    if (pid < 0) exit(1);
    if (pid > 0) exit(0); // Parent exits

    if (setsid() < 0) exit(1);

    signal(SIGTERM, signal_handler);

    // Close standard file descriptors
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);

    // Open syslog
    openlog("my_daemon", LOG_PID, LOG_DAEMON);
    syslog(LOG_INFO, "My daemon started");

    // Main daemon loop
    while (1) {
        syslog(LOG_INFO, "Daemon is running...");
        sleep(30);
    }
}
```

```
    return 0;
}
```

Compile:

```
gcc -o my_daemon my_daemon.c
```

Buat service file `/etc/systemd/system/my_daemon.service`:

```
[Unit]
Description=My Custom Daemon
After=network.target

[Service]
Type=forking
ExecStart=/path/to/my_daemon
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Kelola service:

```
sudo systemctl daemon-reload
sudo systemctl start my_daemon
sudo systemctl status my_daemon
sudo journalctl -u my_daemon
sudo systemctl stop my_daemon
```

5. Linkers and Loaders

Praktik 5.1: Proses Linking

Buat tiga file:

math_ops.h:

```
#ifndef MATH_OPS_H
#define MATH_OPS_H

int add(int a, int b);
int multiply(int a, int b);

#endif
```

math_ops.c:

```
#include "math_ops.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

main.c:

```
#include <stdio.h>
#include "math_ops.h"

int main() {
    printf("5 + 3 = %d\n", add(5, 3));
    printf("5 * 3 = %d\n", multiply(5, 3));
    return 0;
}
```

Proses kompilasi bertahap:

```
# 1. Preprocessing
gcc -E main.c -o main.i
gcc -E math_ops.c -o math_ops.i

# 2. Compilation (assembly)
gcc -S main.c -o main.s
gcc -S math_ops.c -o math_ops.s

# 3. Assembly (object files)
gcc -c main.c -o main.o
gcc -c math_ops.c -o math_ops.o

# 4. Linking
gcc main.o math_ops.o -o program

# Lihat symbols dalam object file
nm main.o
nm math_ops.o
nm program

# Lihat dependencies
ldd program
```

Praktik 5.2: Static vs Dynamic Linking

```
# Static linking
gcc main.c math_ops.c -o program_static -static

# Dynamic linking (default)
gcc main.c math_ops.c -o program_dynamic

# Bandingkan ukuran
ls -lh program_static program_dynamic

# Lihat dependencies
ldd program_static
ldd program_dynamic
```

Praktik 5.3: Membuat dan Menggunakan Shared Library

```
# Compile sebagai shared library
gcc -c -fPIC math_ops.c -o math_ops.o
gcc -shared -o libmathops.so math_ops.o

# Compile program yang menggunakan library
gcc main.c -L. -lmathops -o program_shared

# Jalankan dengan LD_LIBRARY_PATH
LD_LIBRARY_PATH=. ./program_shared

# Lihat informasi loader
readelf -d program_shared
```

6. Why Applications Are Operating-System Specific

Praktik 6.1: System Call Differences

Buat file `os_specific.c`:

```
#include <stdio.h>

#ifdef __linux__
#include <unistd.h>
#include <sys/utsname.h>

void print_os_info() {
    struct utsname info;
    uname(&info);
```

```
        printf("OS: Linux\n");
        printf("System: %s\n", info.sysname);
        printf("Release: %s\n", info.release);
        printf("Version: %s\n", info.version);
    }
#elif _WIN32
    #include <windows.h>

    void print_os_info() {
        printf("OS: Windows\n");
        // Windows-specific code
    }
#else
    void print_os_info() {
        printf("Unknown OS\n");
    }
#endif

int main() {
    printf("=== OS-Specific Application ===\n");
    print_os_info();

    #ifdef __linux__
        printf("\nLinux-specific: Process ID = %d\n", getpid());
    #endif

    return 0;
}
```

Praktik 6.2: Binary Format Analysis

```
# Lihat format executable
file program

# Lihat header ELF
readelf -h program

# Lihat section headers
readelf -S program

# Lihat program headers
readelf -l program

# Disassemble
objdump -d program | head -50
```

7. OS Design and Implementation

Praktik 7.1: Kernel Module Sederhana

Buat file `hello_module.c`:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Student");
MODULE_DESCRIPTION("A simple Linux kernel module");
MODULE_VERSION("1.0");

static int __init hello_init(void) {
    printk(KERN_INFO "Hello Module: Loaded\n");
    return 0;
}

static void __exit hello_exit(void) {
    printk(KERN_INFO "Hello Module: Unloaded\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Makefile:

```
obj-m += hello_module.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Compile dan load:

```
make
sudo insmod hello_module.ko
lsmod | grep hello
dmesg | tail
sudo rmmod hello_module
dmesg | tail
```

Praktik 7.2: Monitoring Kernel

```
# Lihat informasi kernel
uname -r
```

```
cat /proc/version

# Lihat parameter kernel
sysctl -a | head -20

# Lihat kernel ring buffer
dmesg | tail -30

# Lihat kernel modules
lsmod

# Informasi modul tertentu
modinfo ext4
```

8. Operating-System Structure

Praktik 8.1: Eksplorasi Struktur Layering

```
# Layer 1: Hardware
lspci
lsusb
cat /proc/cpuinfo

# Layer 2: Kernel
uname -a
cat /proc/sys/kernel/ostype

# Layer 3: System Calls
strace ls

# Layer 4: System Programs
ps aux
top

# Layer 5: User Applications
which bash
which gcc
```

Praktik 8.2: Microkernel vs Monolithic

```
# Lihat ukuran kernel (Monolithic)
ls -lh /boot/vmlinuz-$(uname -r)

# Lihat built-in vs module
cat /boot/config-$(uname -r) | grep "=y" | wc -l    # Built-in
cat /boot/config-$(uname -r) | grep "=m" | wc -l    # Modules
```

```
# Lihat loaded modules
lsmod | wc -l
```

Praktik 8.3: Program - Virtual File System

Buat file `procfs_read.c`:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char buffer[256];

    // Baca informasi CPU dari /proc
    printf("=== CPU Information ===\n");
    fp = fopen("/proc/cpuinfo", "r");
    if (fp != NULL) {
        int line_count = 0;
        while (fgets(buffer, sizeof(buffer), fp) != NULL && line_count++ < 10) {
            printf("%s", buffer);
        }
        fclose(fp);
    }

    // Baca informasi memori
    printf("\n=== Memory Information ===\n");
    fp = fopen("/proc/meminfo", "r");
    if (fp != NULL) {
        int line_count = 0;
        while (fgets(buffer, sizeof(buffer), fp) != NULL && line_count++ < 5) {
            printf("%s", buffer);
        }
        fclose(fp);
    }

    // Baca informasi process sendiri
    printf("\n=== Process Information ===\n");
    char proc_path[100];
    snprintf(proc_path, sizeof(proc_path), "/proc/%d/status", getpid());
    fp = fopen(proc_path, "r");
    if (fp != NULL) {
        int line_count = 0;
        while (fgets(buffer, sizeof(buffer), fp) != NULL && line_count++ < 10) {
            printf("%s", buffer);
        }
        fclose(fp);
    }

    return 0;
}
```

Compile dan jalankan:

```
gcc -o procfs_read procfs_read.c
./procfs_read
```

9. Building and Booting an Operating System

Praktik 9.1: Analisis Boot Process

```
# Lihat boot messages
dmesg | less

# Lihat systemd boot analysis
systemd-analyze
systemd-analyze blame
systemd-analyze critical-chain

# Lihat GRUB configuration
cat /boot/grub/grub.cfg | head -50

# Lihat initramfs
lsinitramfs /boot/initrd.img-$(uname -r) | head -20
```

Praktik 9.2: Kernel Parameters

```
# Lihat kernel parameters saat boot
cat /proc/cmdline

# Edit GRUB untuk menambah parameter (temporary)
# Saat boot, tekan 'e' di GRUB menu, tambahkan parameter

# Permanent: Edit /etc/default/grub
sudo nano /etc/default/grub
# Tambahkan parameter di GRUB_CMDLINE_LINUX_DEFAULT
# sudo update-grub
```

Praktik 9.3: Simple Bootloader (Konsep)

Buat file `bootloader_concept.asm` (untuk pembelajaran):

```
; Simple bootloader concept (16-bit real mode)
; Tidak untuk dijalankan langsung, hanya untuk pembelajaran
```

```
[BITS 16]
[ORG 0x7C00]

start:
    mov si, msg
    call print_string

    jmp $                ; Infinite loop

print_string:
    lodsb
    or al, al
    jz done
    mov ah, 0x0E
    int 0x10
    jmp print_string
done:
    ret

msg db 'Hello from Bootloader!', 0

times 510-($-$$) db 0    ; Padding
dw 0xAA55                ; Boot signature
```

10. Operating-System Debugging

Praktik 10.1: Using strace

```
# Trace system calls
strace ls

# Trace dengan detail
strace -v ls

# Hitung waktu system calls
strace -c ls

# Trace process yang sedang berjalan
# strace -p <PID>

# Trace dan simpan ke file
strace -o trace.txt ls
cat trace.txt
```

Praktik 10.2: Using ltrace

```
# Trace library calls
ltrace ls
```

```
# Dengan detail
ltrace -c ls
```

Praktik 10.3: Using GDB

Buat program dengan bug `buggy.c`:

```
#include <stdio.h>
#include <stdlib.h>

void buggy_function(int n) {
    int *array = (int*)malloc(n * sizeof(int));

    for (int i = 0; i <= n; i++) { // Bug: should be i < n
        array[i] = i * 2;
    }

    for (int i = 0; i < n; i++) {
        printf("array[%d] = %d\n", i, array[i]);
    }

    free(array);
}

int main() {
    buggy_function(5);
    return 0;
}
```

Debug dengan GDB:

```
# Compile dengan debug symbols
gcc -g -o buggy buggy.c

# Jalankan dengan GDB
gdb ./buggy

# Di dalam GDB:
(gdb) break main
(gdb) run
(gdb) next
(gdb) step
(gdb) print n
(gdb) continue
(gdb) backtrace
(gdb) quit
```

Praktik 10.4: Kernel Debugging dengan dmesg

```
# Lihat kernel log
dmesg

# Follow kernel log
dmesg -w

# Filter by level
dmesg -l err
dmesg -l warn

# Clear ring buffer (requires root)
# sudo dmesg -C
```

Praktik 10.5: Memory Leak Detection dengan Valgrind

```
# Install valgrind
sudo apt install valgrind

# Buat program dengan memory leak
cat > memleak.c << 'EOF'
#include <stdlib.h>

int main() {
    int *ptr = (int*)malloc(100 * sizeof(int));
    // Forgot to free(ptr)
    return 0;
}
EOF

gcc -g -o memleak memleak.c

# Jalankan dengan valgrind
valgrind --leak-check=full ./memleak
```

Praktik 10.6: Performance Analysis

```
# CPU profiling dengan perf
sudo apt install linux-tools-common linux-tools-generic

# Record performance data
# sudo perf record -a sleep 10
# sudo perf report

# Monitor system calls
sudo perf trace ls
```

```
# Top-like interface
sudo perf top
```

Tugas Praktik Komprehensif

Berikut adalah rancangan tugas praktikum komprehensif terkait struktur sistem operasi. Setiap tugas dirancang dengan instruksi spesifik yang memerlukan pemahaman mendalam tentang konsep dasar sistem operasi (seperti process management, file system, shell internals, dan system monitoring). Setiap tugas mencakup elemen-elemen berikut:

- **Implementasi manual di level low-level:** Menggunakan bahasa C (atau C++ jika diizinkan, tapi prioritas C untuk akses langsung ke system calls). Mahasiswa harus menggunakan system calls POSIX/Linux secara eksplisit, tanpa library high-level yang menyederhanakan (misalnya, tidak boleh menggunakan boost atau std::thread untuk process).
- **Dokumentasi mendalam:** Mahasiswa harus menjelaskan setiap baris kode penting dalam komentar, termasuk alasan penggunaan system call tertentu, potensi error, dan bagaimana konsep OS diterapkan. Selain itu, harus ada laporan tertulis yang mencakup diagram alur proses (dibuat manual, bukan generated), penjelasan konsep dasar, dan catatan debugging (error yang ditemui dan cara memperbaiki).
- **Variasi unik berdasarkan NIM:** Setiap mahasiswa harus memodifikasi tugas berdasarkan digit terakhir NIM mereka (misalnya, jika NIM berakhir dengan 1, tambahkan fitur X; jika 2, fitur Y).
- **Pengujian manual dan bukti:** Mahasiswa harus menyertakan screenshot atau log dari pengujian di environment Linux (misalnya, Ubuntu), termasuk perintah compile dan run, serta output yang divalidasi manual.
- **Larangan penggunaan AI:** Secara eksplisit dinyatakan bahwa penggunaan AI untuk generate kode akan dideteksi melalui plagiarisme checker dan wawancara oral untuk menjelaskan kode.

Tugas harus dikerjakan individu, dikumpul dalam bentuk zip file berisi source code, laporan PDF, dan bukti pengujian. Bahasa pemrograman: C (kompilasi dengan gcc). Environment: Linux (rekomendasi Ubuntu 20.04+).

Tugas 1: Process Manager

Deskripsi Umum: Buat program sederhana yang mengelola process menggunakan konsep forking dan inter-process communication (IPC) dasar. Program ini harus mendemonstrasikan pemahaman tentang process creation, synchronization, dan komunikasi.

Instruksi Spesifik:

1. Implementasi:

- Gunakan system call `fork()` untuk membuat minimal 3 child process dari parent process.
- Setiap child process harus menjalankan tugas berbeda: Child 1 menghitung faktorial dari angka input user; Child 2 menghitung bilangan prima hingga batas tertentu; Child 3 membaca string dari user dan membalikinya.
- Parent process harus menggunakan `waitpid()` untuk menunggu setiap child selesai, lalu tampilkan PID child, status exit (gunakan `WIFEXITED` dan `WEXITSTATUS`), dan waktu eksekusi (hitung manual menggunakan `gettimeofday()`).

- Gunakan pipe (system call `pipe()`) untuk komunikasi: Setiap child mengirim hasil komputasinya ke parent melalui pipe unik per child. Parent membaca dari pipe dan menampilkan hasil.
- **Variasi unik:** Berdasarkan digit terakhir NIM mahasiswa:
 - Jika genap (0,2,4,6,8): Tambahkan signal handling di parent untuk menangani SIGINT (Ctrl+C) agar gracefully terminate semua child.
 - Jika ganjil (1,3,5,7,9): Tambahkan shared memory (gunakan `shmget()` dan `shmat()`) sebagai alternatif komunikasi untuk satu child.

2. Dokumentasi:

- Dalam kode: Komentar di setiap system call, jelaskan mengapa digunakan (misalnya, "fork() digunakan untuk membuat process baru karena...").
- Laporan: Gambar diagram process tree (parent dan child), jelaskan konsep zombie process dan bagaimana dihindari, serta catat minimal 2 error yang ditemui selama debugging (misalnya, pipe error) beserta solusinya.

3. Pengujian:

- Compile dengan `gcc -o process_manager main.c`.
- Jalankan dan input data manual (misalnya, angka 5 untuk faktorial).
- Sertakan log output dan strace output (`strace ./process_manager`) untuk membuktikan penggunaan system calls.
- Larangan: Tidak boleh menggunakan `exec()` atau library seperti `pthread`; semuanya manual system calls.

Tujuan Konsep Dasar: Mahasiswa harus paham process lifecycle, IPC, dan synchronization tanpa bergantung pada abstraksi high-level.

Tugas 2: File Monitor

Deskripsi Umum: Buat program yang memantau perubahan pada direktori tertentu dan mencatat aktivitas file, menggunakan system calls untuk file operations.

Instruksi Spesifik:

1. Implementasi:

- Gunakan `inotify_init()`, `inotify_add_watch()` untuk memantau direktori yang ditentukan user (misalnya, `/tmp/monitor_dir`).
- Monitor event: `IN_CREATE`, `IN_MODIFY`, `IN_DELETE`.
- Untuk setiap event, log ke file teks (gunakan `open()`, `write()`) dengan format: [Timestamp] [Event Type] [File Name] [PID Process yang Melakukan].
- Timestamp dihitung manual menggunakan `time()` dan `localtime()`.
- Program berjalan infinite loop hingga di-terminate dengan `SIGTERM`, dan handle signal untuk clean up (tutup inotify fd).
- **Variasi unik:** Berdasarkan digit terakhir NIM:
 - Jika genap: Tambahkan filter untuk hanya log file dengan ekstensi `.txt` atau `.log`.
 - Jika ganjil: Tambahkan notifikasi via email dummy (cetak pesan simulasi, bukan kirim sungguhan).

2. Dokumentasi:

- Dalam kode: Komentar jelaskan struktur `inotify_event` dan bagaimana membaca buffer.
- Laporan: Jelaskan konsep file descriptor dan polling vs. event-driven, gambar flowchart loop monitoring, serta catat error seperti `ENOSPC` pada inotify dan cara handle.

3. Pengujian:

- Buat direktori test, jalankan program, lalu buat/ubah/hapus file manual di terminal lain.
- Sertakan isi log file dan screenshot before/after perubahan direktori.
- Larangan: Tidak boleh menggunakan library seperti libnotify atau fnotify; hanya system calls inotify.

Tujuan Konsep Dasar: Mahasiswa memahami file system monitoring, event handling, dan I/O operations di level kernel.

Tugas 3: Custom Shell

Deskripsi Umum: Tingkatkan simple shell dengan fitur lanjutan, mendemonstrasikan parsing command dan process management.

Instruksi Spesifik:

1. Implementasi:

- Shell dasar: Baca input user dengan `fgets()`, parse command dengan `strtok()` manual (tidak boleh regex library).
- Dukung background process: Jika command diakhiri '&', fork dan jalankan tanpa wait.
- Piping: Dukung single pipe (misalnya, `ls | grep`), gunakan `dup2()` untuk redirect stdin/stdout.
- Redirection: Dukung `>`, `<`, `>>` untuk file I/O.
- Built-in: `cd` (gunakan `chdir()`), `exit`, `pwd` (gunakan `getcwd()`), `echo`.
- Handle error seperti command not found dengan `execvp()`.
- **Variasi unik:** Berdasarkan digit terakhir NIM:
 - Jika genap: Tambahkan built-in 'history' untuk tampilkan 5 command terakhir (simpan di array manual).
 - Jika ganjil: Tambahkan support untuk multiple pipes (minimal 2 pipes).

2. Dokumentasi:

- Dalam kode: Komentar parsing logic, jelaskan bagaimana pipe mengubah file descriptor.
- Laporan: Gambar diagram execution flow untuk piping/redirection, jelaskan konsep foreground vs. background process, catat minimal 3 bug yang ditemui (misalnya, pipe leak) dan fix-nya.

3. Pengujian:

- Jalankan shell, test command seperti `"ls | grep txt > output.txt &".`
- Sertakan transcript session (copy-paste output) dan valgrind report untuk cek memory leak (`valgrind ./myshell`).
- Larangan: Tidak boleh menggunakan readline library; parsing manual sepenuhnya.

Tujuan Konsep Dasar: Mahasiswa paham shell internals, command execution, dan I/O redirection.

Tugas 4: System Information Tool

Deskripsi Umum: Buat tool yang mengumpulkan dan menampilkan informasi sistem menggunakan system calls dan file `proc`.

Instruksi Spesifik:

1. Implementasi:

- CPU info: Baca `/proc/cpuinfo` dengan `open()` dan `read()`, parse untuk tampilkan model, cores, frequency.
- Memory usage: Baca `/proc/meminfo`, hitung used/free manual.
- Disk usage: Gunakan `statvfs()` untuk root filesystem, tampilkan total/used/free.
- Running processes: Baca direktori `/proc`, list PID, nama process (dari `/proc/PID/comm`), status.
- Network interfaces: Baca `/proc/net/dev`, tampilkan interface name, bytes sent/received.
- Output dalam format tabel ASCII (cetak manual dengan `printf`).
- **Variasi unik:** Berdasarkan digit terakhir NIM:
 - Jika genap: Tambahkan CPU load average dari `/proc/loadavg`.
 - Jika ganjil: Tambahkan network socket info dari `/proc/net/tcp` (parse hex address).

2. Dokumentasi:

- Dalam kode: Komentar parsing file `proc`, jelaskan struktur `/proc` filesystem.
- Laporan: Jelaskan virtual filesystem di Linux, gambar hirarki `/proc`, catat challenge seperti permission denied dan handle dengan error checking.

3. Pengujian:

- Jalankan di Linux, sertakan output lengkap dan perbandingan dengan command seperti `top/free/df/ifconfig`.
- Larangan: Tidak boleh menggunakan library seperti `sysinfo` atau `procpfs`; baca file manual.

Tujuan Konsep Dasar: Mahasiswa memahami kernel interfaces dan system monitoring melalui `/proc`.

Referensi dan Resources

Dokumentasi

```
man syscalls      # List of system calls
man 2 fork         # fork system call
man 3 printf       # printf library function
man 5 proc         # /proc filesystem
man systemd.service # systemd service units
```

File-file Penting

- `/proc/*` - Process and system information
- `/sys/*` - Device and driver information
- `/etc/systemd/system/` - Systemd service files
- `/var/log/` - System logs
- `/boot/` - Kernel and boot files

Tools Debugging

- `strace` - Trace system calls
- `ltrace` - Trace library calls
- `gdb` - GNU Debugger
- `valgrind` - Memory debugging
- `perf` - Performance analysis

- `dmesg` - Kernel ring buffer

Monitoring Tools

- `top/htop` - Process monitor
 - `iostat` - I/O statistics
 - `vmstat` - Virtual memory statistics
 - `netstat/ss` - Network statistics
 - `lsof` - List open files
-

Sumber Belajar:

- Operating System Concepts (Silberschatz)
- Linux man pages
- Kernel.org documentation
- GNU C Library documentation